Xijing Wang (Thomas)
Team 6
Teammates: Dillon Ma, Mithun Roy, Leena Aljurashi
ILR09
Apr 7, 2026

Individual Process

This week, my primary contributions for System Demo 7 (dress rehearsal) centered on two areas: (1) implementing and validating the in-place yaw rotation gait for the hexapod in both the MuJoCo physics simulation and the IK engine, and (2) overhauling the Raspberry Pi web-based control panel with streamlined hosting workflows, Xbox controller integration, and camera calibration support for the automated mode.

Robot Rotation Gait Implementation

A critical capability for the hexapod's dance routine is the ability to rotate in place — turning the body about its vertical axis without translating. I implemented this as an extension to the existing tripod gait system in `ik_engine/gait.py`, adding the `rotation_foot_pos()` function that computes foot positions during in-place yaw rotation.

The rotation gait reuses the same tripod coordination as the linear walking gait (legs 0, 2, 4 in Group A alternating with legs 1, 3, 5 in Group B), but instead of translating each foot along a linear stride, it rotates each foot's default position around the body center by a small angular offset. The core math is:

Listing 1: In-place yaw rotation foot position (`gait.py`).

```
def rotation_foot_pos(phase, yaw_stride_rad, lift_h_mm,
                      default_bx, default_by):
    angle_offset, dz = step_pos(phase, yaw_stride_rad,
                                lift_h=lift_h_mm)
    ca = math.cos(angle_offset)
    sa = math.sin(angle_offset)
    bx = default_bx * ca - default_by * sa
    by = default_bx * sa + default_by * ca
    return (bx, by, dz)
```

The key insight is that the existing `step_pos()` function, which generates a smooth stance/swing trajectory for linear walking, can be reused for rotation by treating its output as an angular offset rather than a linear displacement. During the stance phase ($0 \leq p < 0.5$), the foot sweeps along an arc on the ground; during the swing phase ($0.5 \leq p < 1.0$), the foot lifts, swings to its new target position, and sets back down. The lift height is proportional to the foot's radial distance from the body center and the yaw stride angle, ensuring that feet farther from the center lift higher to clear the ground during larger rotations.

MuJoCo Simulation Validation

I integrated the rotation gait into the MuJoCo simulation (`ik_engine/mujoco_sim/demo.py`) to validate the motion before deploying to hardware. The simulation supports a `--script-rotate` command-line argument that accepts a rotation command in $[-1, 1]$, along with `--yaw-stride` to configure the maximum yaw per step (default 10°). The control logic in the simulation distinguishes between linear translation and in-place rotation by comparing the magnitudes of the forward/side speed and the rotation command:

Listing 2: Rotation vs. translation decision in MuJoCo sim.

```
if rot_speed > speed:
    # In-place yaw rotation (tripod)
```

```
    yaw_rad = math.radians(rotate * yaw_stride_deg)
    radius = math.hypot(bx0, by0)
    lift_h = radius * abs(yaw_rad) * lift_scale
    new_bx, new_by, dz = rotation_foot_pos(
        phase, yaw_rad, lift_h, bx0, by0)
    robot.set_foot_body(leg_index, new_bx, new_by,
                        config.default_foot_z + dz)
```

Figure 1 shows a top-down view of the hexapod during the rotation gait, visualized using the IK engine's forward kinematics. The alternating tripod groups are visible: Group A legs (0, 2, 4) and Group B legs (1, 3, 5) alternate between stance (ground contact) and swing (lifted) phases as the robot rotates about its body center.

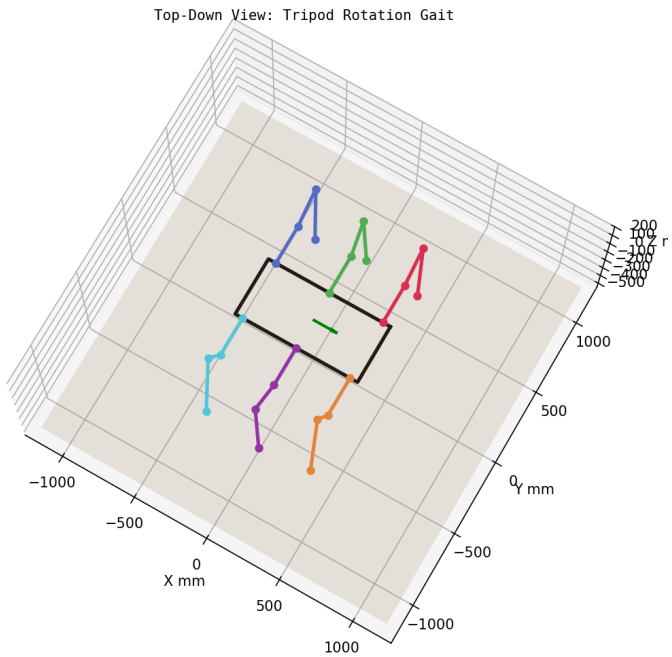

Figure 1: Top-down view of the tripod rotation gait. The six legs are positioned at their mid-rotation targets, showing the circular arc each foot follows during in-place yaw. Group A (legs 0, 2, 4) and Group B (legs 1, 3, 5) alternate between stance and swing phases.

The simulation also supports interactive keyboard control (`.` and `/` keys for turn left/right) and scripted presets (`--script-preset turn-left` or `turn-right`), as well as GIF recording via `--save-gif` for documentation and debugging. This rotation capability is essential for the dance routine, enabling the hexapod to perform turns as part of choreographed motion sequences.

Raspberry Pi Control Panel Overhaul

The second major contribution was a significant update to the web-based control panel (`robot/web/`), which runs on the Raspberry Pi and provides a browser-based interface for controlling the hexapod. The overhaul focused on three areas: streamlined hosting, Xbox controller integration, and auto-mode camera calibration.

Dashboard and One-Click Hosting

The dashboard (`robot/web/templates/dashboard.html`) now provides a unified overview of all robot subsystems — runtime status, camera status, and Xbox controller status — with one-click deployment buttons. The "Deploy Manual Mode" button starts the motion runtime and Xbox controller together, while "Deploy Automated Mode" starts the runtime and camera detection service simultaneously. This eliminates the need to manually SSH into the Pi and start each service individually.

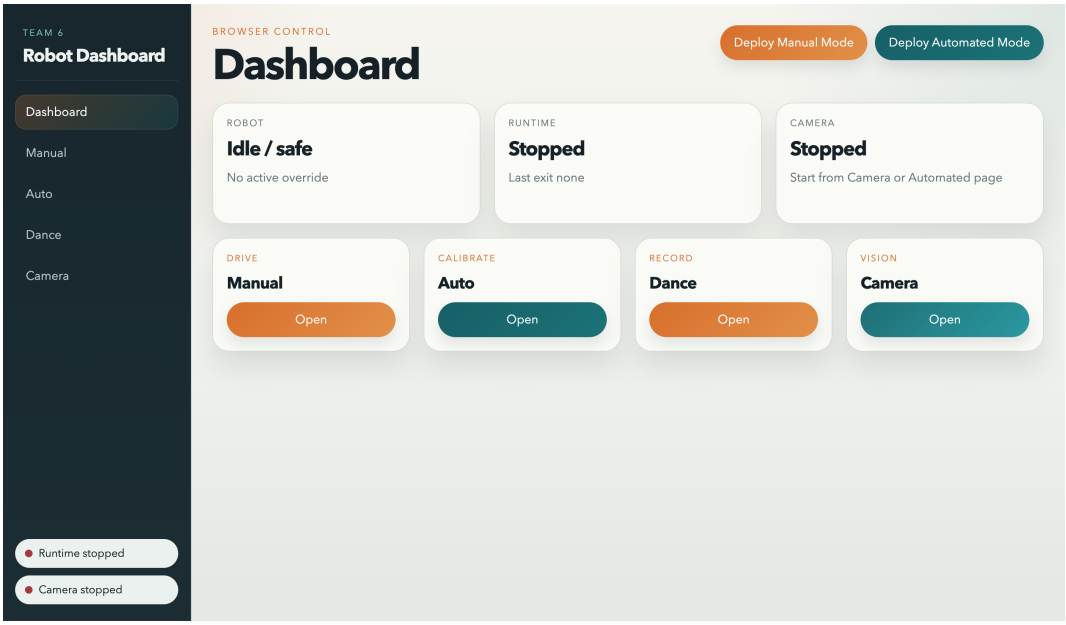

Figure 2: Updated Robot Dashboard with one-click deployment buttons for Manual and Automated modes. Status cards show runtime, camera, and override state at a glance.

The hosting logic is managed by the `ManagedProcessManager` class in `process_control.py`, which handles process lifecycle (start, stop, health monitoring) for three independent services:

- **Motion Runtime:** The IK gait controller and UART bridge (`motion_engine/runtime/main.py`).
- **Camera Service:** The vision pipeline for AprilTag detection (`vision/live.py`).
- **Xbox Controller:** The gamepad input bridge (`motion_engine/runtime/xbox_controller.py`).

Each service is configured via a typed dataclass (`RuntimeLaunchConfig`, `CameraLaunchConfig`, `XboxControllerLaunchConfig`) that defines command-line arguments, ports, and defaults. The web API endpoints (`/api/deploy/manual`, `/api/deploy/automated`, `/api/deploy/stop-all`) orchestrate starting and stopping these services in the correct order.

Manual Mode and Xbox Controller Integration

The Manual Control page (`manual_mode.html`) provides keyboard-style directional controls (Q/W/E/A/S/D mapped to turn-left, forward, turn-right, strafe-left, backward, strafe-right) that send override commands to the motion runtime via the `OverrideStore`. For System Demo 7, I integrated the Xbox controller as an alternative input source: the "Deploy Manual Mode" button now automatically starts the Xbox controller process alongside the motion runtime.

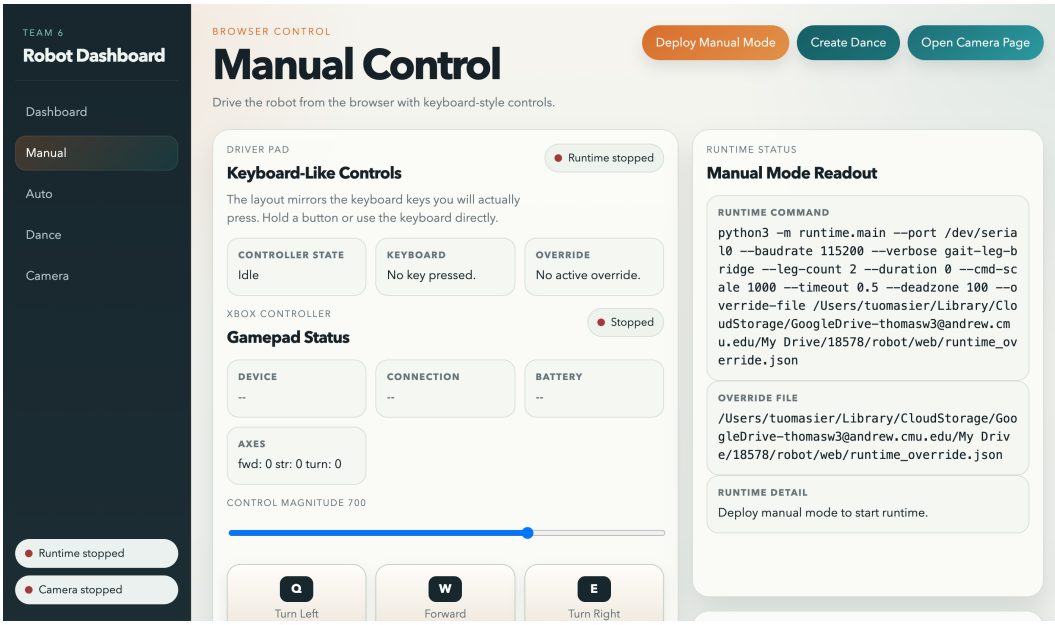

Figure 3: Manual Control page showing keyboard controls, Xbox gamepad status section (device, connection, battery, axes), and the runtime command preview with override file path.

The Xbox controller status panel displays real-time connection state, battery level, and axis values (`fwd`, `str`, `turn`), polled via the `read_xbox_status()` function. The gamepad input is forwarded to the motion runtime through the same override mechanism as the keyboard controls, ensuring consistent behavior regardless of input source. A configurable "Control Magnitude" slider adjusts the command strength sent from the web UI, independent of the Xbox controller's own analog input range.

Auto Mode and Camera Calibration

The Auto Mode page (`auto_mode.html`) implements a guided calibration workflow for the hexapod's autonomous operation. The three-step process is:

1. **Deploy Automated Mode:** One-click starts the motion runtime and the AprilTag detection camera service together.
2. **Check Calibration:** The page polls the camera service's `/detections` endpoint and displays real-time tag count, tag IDs, and calibration state. The camera stream is embedded directly in the page as an MJPEG stream, allowing the operator to visually verify tag visibility without switching to a separate camera page.
3. **Pick a Dance Preset:** Once calibration is confirmed, the operator can select from predefined dance presets (Groove Alpha, Shuffle Beta, Spin Gamma) that will drive the choreography engine.

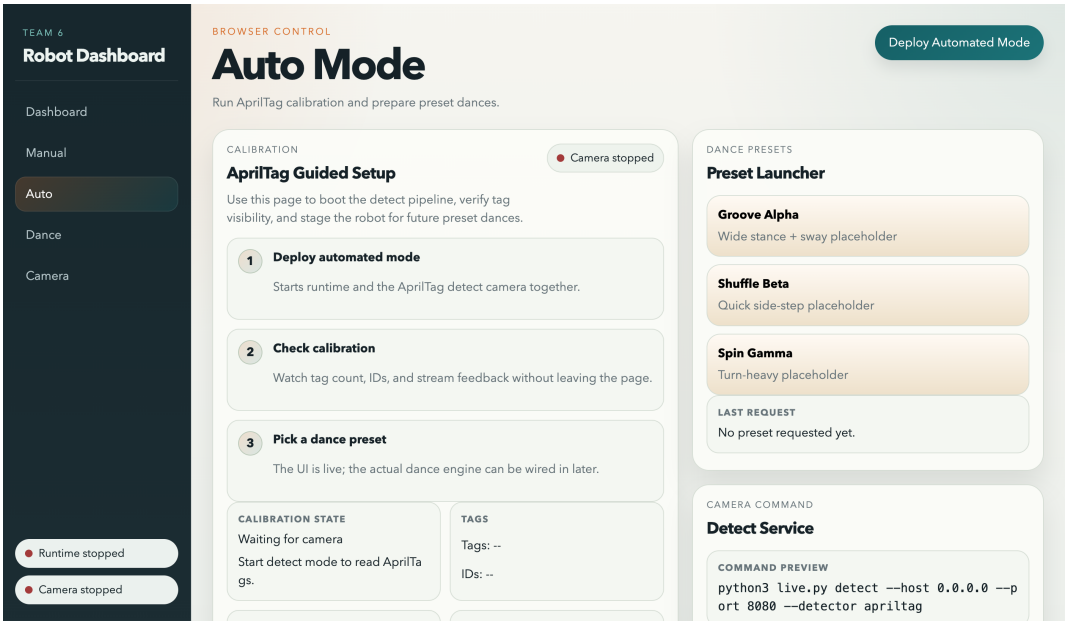

Figure 4: Auto Mode page with AprilTag calibration workflow. Left: guided setup steps and embedded camera stream. Right: dance preset launcher and camera service configuration.

The camera calibration logic handles both managed and external camera services: if the web dashboard started the camera, it manages the process lifecycle directly; if a camera service was started externally (e.g., via SSH), the dashboard detects it by probing the health endpoint and displays its status without attempting to manage the process. This dual-mode approach ensures the dashboard works correctly regardless of how the system was deployed.

Challenges

The primary challenge was determining the correct rotation strategy for the hexapod. Unlike linear walking, in-place yaw rotation requires each foot to follow a circular arc rather than a linear stride, and the lift height must scale with the foot's radial distance from the body center to prevent dragging. I researched various hexapod rotation approaches by watching videos of other hexapod robots and studying their gait patterns. Many designs use alternative strategies such as body-frame rotation (keeping feet planted and rotating the body) or wave gaits with sequential leg repositioning. After reviewing these options, I concluded that the tripod-based rotation — where the same alternating three-leg groups used for walking also coordinate the rotation — was the most natural fit for our existing gait system. This approach reuses the proven tripod coordination and the `step_pos()` trajectory generator, requiring only a reinterpretation of the linear stride as an angular offset. The result is a rotation gait that integrates seamlessly with the walking gait and can transition smoothly between the two.

A secondary challenge was ensuring the web control panel works correctly on both the Raspberry Pi (the deployment target) and a development laptop (for testing and UI iteration). The process management code (`process_control.py`) references hardware-specific paths and commands (e.g., UART device paths, camera device files) that do not exist on a development machine. I structured the `ManagedProcessManager` so that the web UI renders and responds correctly even when

the underlying hardware processes fail to start, displaying appropriate error messages instead of crashing.

Team Work

The team continued to make progress on hardware and software integration for the dress rehearsal:

- **Thomas (me):** Implemented in-place yaw rotation gait in the IK engine and MuJoCo simulation. Overhauled the Raspberry Pi web control panel with one-click hosting for manual and automated modes, Xbox controller integration, and camera calibration workflow for auto mode. AprilTag calibration integration in the IK engine has not yet been started.
- **Dillon:** Completed assembly and mounting of four out of six legs on the chassis, with the remaining two legs largely assembled. Full six-leg mounting is expected to be completed before the Final System Demo.
- **Mithun:** Completed the majority of the wiring harness work, routing servo cables through the chassis structure and connecting servos to the PCA9685 driver boards. Some wiring remains to be finalized once the last two legs are fully mounted.
- **Leena:** Assisted Mithun with the wiring harness work, helping route and connect servo cables for the mounted legs.

Plans

For the Final System Demo, the critical remaining tasks are:

- **Complete hardware assembly:** Dillon needs to finish mounting the remaining two legs, and Mithun and Leena need to complete the final wiring connections. This is the critical-path item that gates all subsequent integration testing.
- **Manual mode walking and rotation on hardware:** Once the chassis is fully assembled with all six legs, deploy the walking and rotation gaits to the physical robot and tune parameters (stride length, lift height, gait frequency, yaw stride) so the robot can move reliably under manual control via the Xbox controller or the web dashboard. The goal is for the hexapod to walk forward, backward, strafe, and rotate in place smoothly in manual mode.
- **Auto mode preset dances:** Implement basic preset dance sequences in the automated mode. Each preset is a timed list of motion primitives (`walk_forward`, `turn`, `side_step`) that the gait controller executes in order. The web UI's preset launcher buttons are already functional — they just need to connect to the actual choreography runner on the motion runtime side.
- **AprilTag calibration in IK engine:** Implement the AprilTag-based calibration logic inside the IK engine, using detected tag positions from the camera to calibrate the robot's heading and position before executing a dance sequence. This has not yet been started and is a priority for the final milestone.
- **Coolness factor — dance creator:** As an extra feature, enable users to create custom dance routines through the Dance Creator page on the web dashboard. In manual mode, the operator drives the robot while the system records the sequence of override commands (direction, magnitude, timing) into a dance recording. The recorded session can then be saved and replayed as a new preset dance, effectively encoding manual-mode motion into a reusable choreographed sequence.